DevOps pipeline for a course review web app

DD2482 project report

Georgi Tsonchev Hristov (georgith@kth.se), Cesar Aceves Hernández (cesarah@kth.se)

1. Introduction

We made a small web app where students can write anonymous reviews of KTH courses. It is made with Flask and SQLite, and it has one page with a form and the list of reviews. The app is simple on purpose, because the project is about the pipeline. Everything is self-hostable, anyone with Docker and Terraform can run the same setup on their own machine.

The code is on GitHub: https://github.com/tctev/course-reviews. The link to the deployed app is in our submission PR.

2. Architecture

The app runs with gunicorn in a Docker container on port 8000. The image uses `python:3.13-slim` and the app does not run as root. The database is the file `/data/reviews.db`. `/data` is a Docker volume, so when we deploy a new version the reviews are not lost.

The parts of the system are:

- GitHub for the code, pull requests, GitHub Actions and the container registry (GHCR).
- A CI workflow that runs on every pull request, on GitHub-hosted runners.
- A CD workflow that runs on every push to `main`. The image is built on a GitHub-hosted runner and the deploy runs on our self-hosted runner.
- Terraform with the `kreuzwerker/docker` provider for the network, the volume, the image and the container.
- An Azure VM (Ubuntu 24.04, Azure for Students) with the self-hosted runner and the app.

When we make a change, we open a PR and CI runs. We can only merge when CI passes. After the merge the image is built, tagged with the commit SHA and pushed to GHCR. Then the self-hosted runner runs `terraform apply` with the new tag and checks that the app works.

3. Continuous integration

The CI workflow runs on every pull request. It installs the dev dependencies and runs:

1. `ruff check` and `ruff format --check` for linting and formatting.
2. `pytest`. The tests post a review, check that bad input is rejected (wrong course code, rating not 1-5, empty or too long text) and check that HTML is escaped.
3. `terraform fmt -check`, `terraform init -backend=false` and `terraform validate`.
4. `docker build`, to find problems in the Dockerfile before merging.

`main` is protected. Every change needs a PR and the `test` job has to pass. This is also for admins, so we can't push to `main` directly. The workflow token only has `contents: read`. We changed this after CodeQL warned us about the `GITHUB_TOKEN` permissions.

4. Continuous deployment

The CD workflow runs on push to `main`. It has two jobs.

The `build` job runs on `ubuntu-latest`. It logs in to GHCR and pushes `ghcr.io/tctev/course-reviews:<commit sha>`. We use the SHA as tag and not `latest`. This way every deploy uses one exact image, and we can see which commit is running. To go back to an old version we just run the apply again with the old SHA.

The `deploy` job runs after `build`, on the self-hosted runner. It logs in to GHCR, installs Terraform, pulls the image and runs `terraform apply -var image=...`. At the end there is a smoke test with `curl` on `localhost:8000`. If the app doesn't answer, the job fails. curl tries again for a few seconds, because sometimes gunicorn is not ready yet. The workflow has `concurrency: deploy`, so only one deploy runs at a time.

The Terraform state is a file in the home folder of the runner user. It can't be in the workspace, because `actions/checkout` cleans the workspace every time. Then Terraform would forget the container and try to create it again.

5. Infrastructure as Code

`terraform/main.tf` has four resources: `docker_network`, `docker_volume` for the database, `docker_image` and `docker_container`. The image is a variable. In CD it is the GHCR image with the SHA. The default is `course-reviews:local`, so anyone with Docker can run `docker build -t course-reviews:local .` and then `terraform apply` and get the same setup on their computer. This was one of the goals in our proposal.

When the image tag changes, Terraform only replaces the container. The network and the volume stay, so the database is kept. The provider version is fixed in `.terraform.lock.hcl`.

6. Quality and security automation

- CodeQL runs on PRs, on pushes to `main` and once a week. It checks the Python code and the workflow files. It found the token permissions problem from section 3, and now there are no open alerts.
- Dependabot checks pip, GitHub Actions, the Docker base image and the Terraform provider every week.
- Ruff and pytest run in CI (section 3).

7. Design decisions

We used GitHub Actions because our code was already on GitHub. This way CI, CD and the registry are in one place, and we don't need our own CI server.

We used Terraform with the Docker provider. Kubernetes or a big cloud setup is too much for one container. With the Docker provider the same file works on a laptop and on a server.

We need a self-hosted runner because Terraform has to talk to Docker on the machine where the app runs. A GitHub-hosted runner is deleted after the job. First the runner was on Georgi's laptop. It worked, but we had to install Docker, Terraform and the runner on a personal laptop, and deploys only worked when the laptop was on. So we moved it to an Azure VM and installed everything there once. The VM is just a normal Linux machine, nothing in the setup needs Azure. The student subscription only has a few regions, so the VM is in Denmark East. It has 1 GB of RAM. This is enough because the image is built on GitHub and the VM only pulls it, but we added 2 GB of swap to be safe.

We used SQLite in a volume because it is just one file and we don't need a database server. For a small app like this it is enough.

8. Limitations and trade-offs

- Everything runs on one VM. If the VM is down, the image is still built and pushed, but the deploy waits for the runner. There is no failover and no staging.
- The Terraform state is a local file, with no locking or backup. In a real project it should be in a remote backend. This was a problem once: an old deploy left a Docker network that was not in the state, so the next `terraform apply` failed and we had to delete the network by hand.
- SQLite doesn't work with more than one copy of the app, so we can't scale.

- The app uses HTTP on port 8000. There is no HTTPS and no reverse proxy.
- The runner user is in the `docker` group, and that is almost like root on the VM. The repository is public, so only deploys from `main` run there. Pull requests, also from forks, run on GitHub-hosted runners. But anyone who can merge to `main` can still run code on the VM.
- We don’t have many tests. They test the form and the input checks. The deploy is only tested by the smoke test.